

Drawing, Text, Shapes

There are basic drawing, shape filling, outlines and text with different graphic colors and styles.

The process of shape rendering is similar to 3D programming using Contexts. Define a context, add shapes, color and styles, and render into an image. Once a context is created arguments and parameters cannot be removed, only replaced. The context will render to an image using PutToImage.

Shapes Creation

Shapes are linked lists of line segments. Define by shape by setting a pointer to Shape(), MoveTo() the start location, and LineTo() all other points in the shape.

Shape

Summary

This creates a shape link list table to add points and line segments.

Usage

Shape()

Example

```
local sh = Shape()
```

AddRectangle

Summary

AddRectangle will create a rectangle shape with round corners.

Usage

shape:AddRectangle (*float* Left, *float* Right, *float* Top, *float* Bottom, *float* Corner Radius, *float* Precision)

- **Left** edge
- **Right** edge
- **Top** edge
- **Bottom** edge
- **Corner Radius** zero is sharp corner
- **Precision** Anti Aliasing oversampling

Example

```
sh:AddRectangle ( -0.1, 0.1, - 0.1, 0.1, 0.01, 8)
```

MoveTo

Summary

This sets a point in X Y space

Usage

shape:MoveTo(number X, number Y)

Example

```
sh:MoveTo(0.101649485528469, -0.175463914871216)
```

LineTo

Summary

This defines a line segment from the previous defined point. Use MoveTo to define the first point.

Usage

shape:LineTo(number X, number Y)

Example

```
sh:MoveTo(0.101649485528469, -0.175463914871216)
sh:LineTo(0.132989693308614, -0.175876285507507)
sh:LineTo(0.127628862857819, -0.19814433157444)
sh:LineTo(0.115670099854469, -0.204742267727852)
sh:LineTo(0.107961280143138, -0.197474031147269)
sh:Close() -- Close shape to the origin point
```

BezierTo

Summary

This defines a Bezier curve point and handles.

Usage

shape:BezierTo(number pointX, number pointY, number handle1X, number handle1Y, number handle2X, number handle2Y)

Example

```
shbz:BezierTo( -0.053, 0.097, 0.000, 0.096, 0.055, 0.092)
shbz:BezierTo( 0.113, -0.004, 0.082, -0.053, 0.057, -0.103)
shbz:BezierTo( -0.055, -0.097, -0.080, -0.048, -0.108, 0.002)
shbz:Close()
```

ConicTo

Summary

This defines a Conic curve point2.

Usage

shape:ConicTo(*number* point1X, *number* point1Y, *number* point2X, *number* point2Y)

Example

```
shcon:ConicTo( -0.053, 0.097, 0.000, 0.096)
shcon:ConicTo( 0.113, -0.004, 0.082, -0.053)
shcon:ConicTo( -0.055, -0.097, -0.080, -0.048)
shbz:Close()
```

Close

Summary

Shapes can be closed from the first point to last point

Usage

Close()

Example

```
sh = Close()
```

Text Shape

GetCharacterShape

Summary

This will get a Font glyph shape and put in a shape object

Usage

Shape =Fontmetrics:GetCharacterShape(string ch, false)

Example

```
sh = tfm:GetCharacterShape(ch, false)
```

TextStyleFont

Summary

The font typeface

Usage

Shape =Fontmetrics:GetCharacterShape(string ch, false)

Example

```
local font = TextStyleFont(font, style)
```

TextStyleFontMetrics

Summary

This will get a Font character and put in a shape object

Usage

Shape =Fontmetrics:GetCharacterShape(*string* ch, false)

Example

```
local tfm = TextStyleFontMetrics(font)
```

CharacterWidth

Summary

This will get a Font character and put in a shape object

Usage

Shape =Fontmetrics:GetCharacterShape(*string* ch, false)

Example

```
cw = tfm:CharacterWidth(ch)*10*size
```

CharacterKerning

Summary

This will get a Font character and put in a shape object

Usage

Shape =Fontmetrics:GetCharacterShape(*string* ch, false)

Example

```
x_offset = tfm:CharacterKerning(prevch, ch)*10*size
```

OutlineOfShape

Summary

This will define the shape as an outline, with a thickness, and joint type to define the elbows at the end of each segment. Windmode defines the method of dealing with overlapping lines. Line type defines whether the outline is solid or broken up as dots and dashes.

Usage

shape sh = shape: OutlineOfShape(*float* thickness, *arg* linetype, *arg* jointype, *integer* precision, *arg* windmode, *integer* flatten precision)

thickness

This defines the thickness of the line, a value of 0.001 is 1/1000 the width of the image.

linetype

- OLT_Solid
- OLT_Dash
- OLT_Dot
- OLT_DashDot
- OLT_DashDotDot

jointype

- OJT_Bevel
- OJT_Miter
- OJT_Round

precision

- Set to 8

windmode

- SWM_NoChange
- SWM_Normal
- SWM_Clear
- SWM_Solid

flattenprecision

- Set to 8

Example

```
sh = sh:OutlineOfShape(thickness, "OLT_Solid", "OJT_Bevel", 8, "SWM_Normal", 8)
```

ImageChannel

Summary

ImageChannel is a monochrome buffer that you can draw shapes on. It is connected to an actual Image object, you just need to call PutToImage(mode, channelstyle) to "bake" the buffer onto it. While doing so, you have the option to overwrite the image (mode = "CM_Copy") or merge the buffer to what's already there ("CM_Merge"). To put the ImageChannel onto the image, Fusion also needs to know what color you'd like to paint in. This is done with a ChannelStyle object which stores attributes like color and opacity but also fills gradient or softness.

This allows you to reuse an ImageChannel (a "drawing") multiple times. By changing the ChannelStyle each time you call PutToImage() you can create a semi-transparent red and a blurry green version, for example.

The color, gradient and softness options at the bottom of the shading tab of the Text+ tool basically expose the things you can do with a ChannelStyle.

A Shape object is one or several vector-based polygons or a collection of lines and beziers. It can be transformed without quality loss since it only gets rendered by calling Image Channel's FillShape() method. The Shape class provides methods to expand or shrink the shape or to create an outline of the polygon you have drawn. In fact, since shapes are always rendered by filling them, you need to create an outline version of the shape with a defined thickness and line style (e.g. dotted or dashed) and then fill this shape to get your outline. These features can be found in most mask tools as well as the Text+ tool.

Usage

ImageChannel(image out, number sampling)

Members

Image	Points to the image for rendering
Sampling	This define the amount of over sampling

Example

```
local ic = ImageChannel(out, 8)
```

Styles

FillStyle

Summary

FillStyle sets up a "look" for rendering shapes into images.

Usage

FillStyle()

Example

```
local fs = FillStyle()
```

SetFillStyle

Summary

SetFillStyle associates the style look to the Image Channel Image

Usage

imageChannel:Image:SetFillStyle(fillstyle)

Example

```
ic:SetStyleFill(fs)
```

ShapeFill

Summary

ShapeFill associates the shape to the Image Channel Image.

Usage

ImageChannel image:ShapeFill(*Shape*)

Example

```
ic:ShapeFill(sh2)
```

PutToImage

Summary

This will render the shape into an image, using the styles set.

Usage

ImageChannel:PutToImage(*table* Attributes, *ChannelStyle* cs)

Attributes

Name	Description
"CM_Merge"	This will merge the rendered shape into the image
"CM_Copy"	This will replace the image with the rendered shape

Example

```
ic:PutToImage("CM_Merge", cs)
```

ChannelStyle

A ChannelStyle object stores properties like color, opacity and softness that are used to render a shape into ImageChannel into an image. This way, you can reuse an image style for multiple calls to ImageChannel:PutToImage().

Summary

Defines a ChannelStyle object. Using the commands to add color and look parameters listed below

Usage

ChannelStyle()

Example

```
local cs = ChannelStyle()
```

Color

Summary

Sets the Color RGBA values to the ChannelStyle object

Usage

ChannelStyle.Color = Pixel {number R, number G, number B, number A}

Example

```
cs.Color = Pixel{R=r , G=g , B=b, A = 1}
```

BlurType

Summary

Sets the type of Blur filter to process ChannelStyle object

Usage

ChannelStyle.BlurType = "Attribute"

Attribute

- "BT_Box"
- "BT_Soften"
- "BT_Bartlett"
- "BT_Sharpen"
- "BT_Gaussian"
- "BT_FastGaussian"
- "BT_Hilight"
- "BT_Blend"
- "BT_Solarise"
- "BT_MultiBox"

Example

cs.BlurType = "BT_Bartlett" -- BT_Box, BT_Bartlett, BT_MultiBox, BT_Gaussian

SoftnessX SoftnessY

Summary

Sets the amount of Blur in the X and Y directions

Usage

ChannelStyle.SoftnessX = number Blur

ChannelStyle.SoftnessY = number Blur

Example

```
cs.SoftnessX = 10.0  
cs.SoftnessY = 10.0
```


SoftnessGlow

Summary

Sets the amount of Glow for the blur. 0.0 will be no glow, 1.0 is full glow, used in conjunction with SoftnessBlur, see the glow tool for behavior.

Usage

ChannelStyle.SoftnessGlow = number Glow

Example

```
cs.SoftnessGlow = 0.95
```

SoftnessBlend

Summary

Sets the amount of Blend between the Glow image and the original image.

Usage

ChannelStyle.SoftnessBlend = number Blur

Example

```
cs.SoftnessBlend = 0.3
```

Shape Transforms

Shapes can be moved, rotated, scaled, sheared and perspective projected via matrix math.

Matrix4

Summary

Matrix4 defines a 4x4matrix which has XYZ position, rotation, and scale

Usage

To create a Matrix4 object, you can call:

```
-- <table> is a table of 16 elements  
mat = Matrix4(<table>)
```

To convert a Matrix4 object back to a table (for printing etc...), call:

```
<table> = mat:GetTable()
```

Most matrix methods modify the existing matrix object and "add" their transformations to it (e.g. Move(), Scale(), ...). The Inverse() and Transpose() methods, however, simply return the desired matrix without modifying the object itself.

The Matrix4 class has overloaded operators in LUA, which means you can multiply and add matrices by simply writing

```
mat1 = Matrix4()  
mat2 = Matrix4()  
mat3 = mat1 * mat2    -- transform defined by mat1 followed by  
                       transform defined by mat2
```

Matrix Operations

Matrix Math and Fusion's Coordinate System

In Fusion's coordinate system, 1.0 denotes full width (e.g. 1920 pixels in HD) but also full height (e.g. 1080 pixels). The coordinate system is squashed. The Matrix4 and Shape objects, however, work with coordinates that are scaled the same way in both directions based on the Width being 1.0.

The horizontal image size was 1920, a Y value of "1" would also stand for 1920 vertical pixels. If you want to move a shape to the coordinates defined by the user using an OffsetControl input, you need to account for this. The formula to convert y-coordinates for use with Matrix4 is:

$$\text{matrix_Y} = \text{fusion_Y} * (\text{img.Height} * \text{img.YScale}) / (\text{img.Width} * \text{img.XScale})$$

(img is the destination image and is used to retrieve the dimensions and pixel aspect ratio)

Identity

Summary

Sets the Matrix to a default state, with position and rotation at 0.0 and scale at 1.0

Usage

matrix:Identity()

Example

```
mat:Identity()
```

RotX RotY RotZ

Summary

These functions will rotate the matrix around each axis X,Y or Z, with the angle defined in degrees.

Usage

matrix:RotX(float angle)

matrix:RotY(float angle)

matrix:RotZ(float angle)

Example

```
mat:RotZ(rotation)
```

RotAxis

Summary

This function will rotate the matrix around a defined axis, with the angle defined in radians.

Usage

matrix:RotAxis(float axisX, float axisY, float axisZ, float radians)

Example

```
mat:RotAxis(0.0, -1.0, 0.0, rotation)
```

Rotate

Summary

This function will rotate the matrix around each axis, with the order of operations set by the Order argument.

Usage

matrix:Rotate(float angleX, float angleY, float angleZ, arg Order)

Arguments

- Order of operations
- RO_ZYX
- RO_YZX
- RO_ZXY
- RO_XZY
- RO_YXZ
- RO_XYZ

Example

```
mat:RotAxis(30.0, -14.0, 10.0, RO_ZXY)
```

Move

Summary

This function will apply a translation to the matrix.

Usage

matrix:Move(float X, float Y, float Z)

Example

```
mat:Move(0.75, 0.15, 0)
```

Scale

Summary

This function will apply a scale to the matrix.

Usage

matrix:Scale(float X, float Y, float Z)

Example

```
mat:Scale(0.75, 0.75, 0.75)
```

Shear

Summary

This function will apply a Shear to the matrix along the XYZ axis

Usage

matrix:Shear(float X, float Y, float Z)

Example

```
mat:Shear(0.75, 0.0, 0.0)
```

Project

Summary

This function will apply 3D perspective to a matrix based on a Field of View

Usage

matrix:Project(float FoV)

Example

```
mat:Project(fov)
```

Perspective

Summary

This function will apply 3D perspective to a matrix based on a Field of View, Aspect of the view window, and the clipping planes.

Usage

matrix:Perspective(float fovy, float aspect, float zNear, float zFar)

Example

```
mat:Perspective(fovy, aspect, zNear, zFar))
```